

Web Application Implementation Report

Group CE

A

Xue Bai 3090581B

Jocelyn Sickler 3112960S

Aasim Ali Khan 3171501K

Introduction (½ Page Max):

KindredConnect is a centralised community platform designed to bridge the gap between elderly residents, local care homes, and volunteers.

It has three intended users:

- Seniors (living at home or in care homes who need assistance or social engagement)
- Care home managers (staff responsible for organising resident events and recruiting volunteers)
- Volunteers (community members looking for opportunities to offer their time and skills)

This app seeks to solve the problem of senior loneliness and limited access to community support. Care homes often lack digital tools to recruit and manage local volunteers efficiently, leaving resources underutilised. By offering a mobile-first, accessible interface, KindredConnect simplifies resource matching, increases senior engagement, and empowers volunteers to contribute effectively to their local communities.

Github Repository Link: <https://github.com/Jocundity/Itech-project>

Deployed Web Application Link: <https://kindredcareconnect.onrender.com>

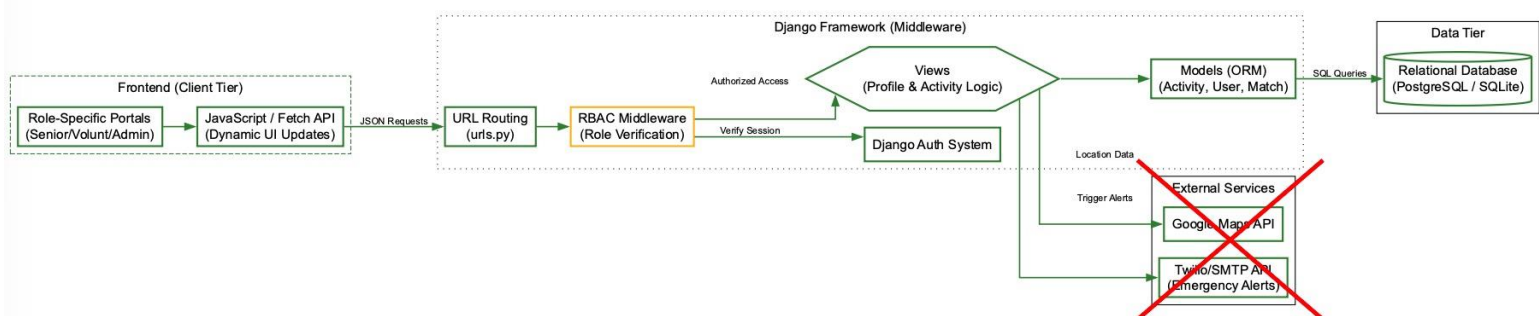
Updated Design Specification (6 Pages Max):

I.) User Stories:

ID	User Story	MOSCOW Priority
M1	As a new user, I want to create an account by selecting my role (Volunteer/Senior/Care Home), so that I can access features specific to my profile.	Must
M2	As a Care Home Manager or Senior, I want to fill in a form to post an activity with a title, date, and location, so that the data is saved in the database.	Must
M3	As a Volunteer, I want to view a list of all available activities, so that I can see	Must

	information fetched from the system's database.	
M4	As a Senior or Care Home Manager, I want to click the 'Accept' button on an activity, so that the system matches me to a volunteer in the database.	Must
S1	As a user (of any type), I want to log in with my email and password, so that I can securely access my personalised dashboard.	Should
S2	As a Volunteer, I want to filter activities by category (e.g. Outdoor Activities), so that I can find tasks that match my interests.	Should
C1	As a Volunteer, I want to receive a confirmation message after I've been approved to take part in an activity, so that I know my request was successful.	Could
C2	As a Senior, I want to see high-contrast text and large buttons, so that the application is easier for me to navigate.	Could

II.) System Architecture Diagram:

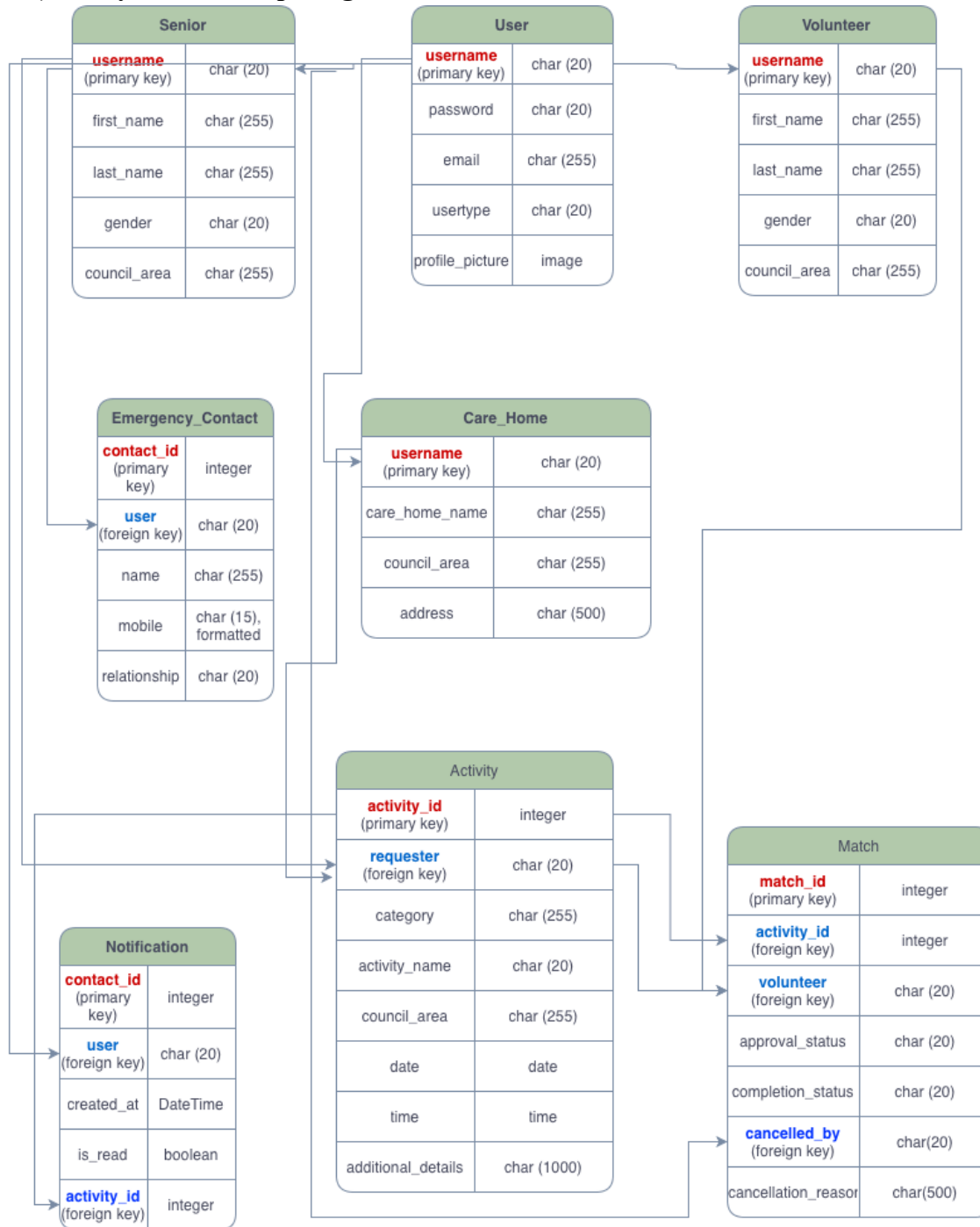


This web application uses HTML templates, CSS stylesheets, and client-side Javascript for the frontend. It uses the Python-based framework, Django, for the middleware, and Django's inbuilt SQLite database as the server/backend layer.

Due to time constraints, we have made the decision to alter our design not to rely on external APIs like Google Maps (for location searching) or Twilio (for Emergency Alerts). We did, however, include an

Emergency Contact form in the user profile for our “Senior” user type, so this functionality can be added at a later date.

III.) Entity-Relationship Diagram:

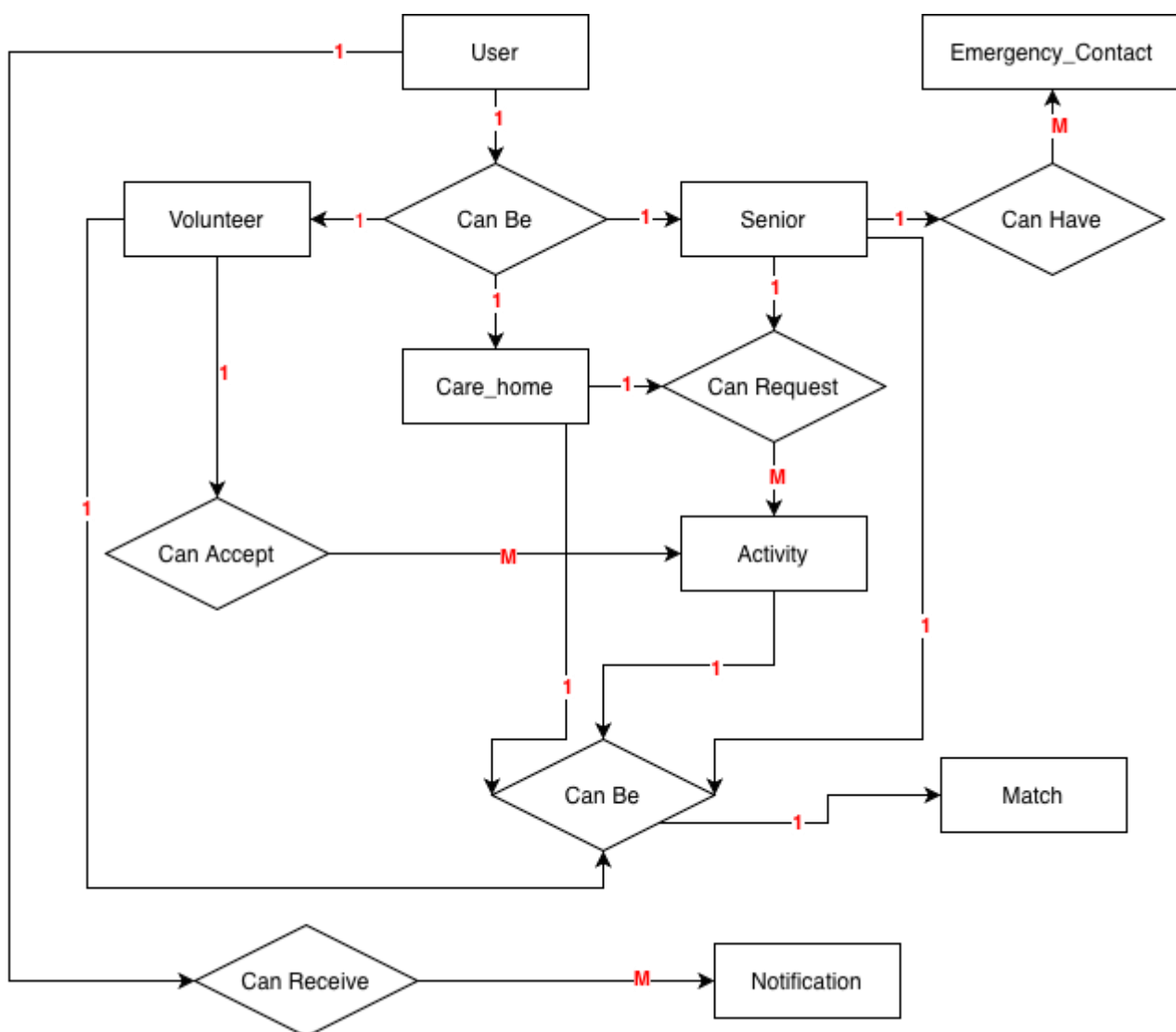


Entities: User, Senior*, Volunteer*, Care Home*, Emergency Contact, Activity, Match, Notification

Primary keys are denoted in red; foreign keys are denoted in blue; relationships are represented with arrows

* Please note that in the django model, all users are stored under the “UserProfile” model. However, they have different signup fields (which is represented in the diagram above) based on their user type and each user type has different functionalities. Therefore, we have decided to display them as different entities in this diagram.

Changes from the Original Design: As mentioned in the note above, we have decided to model all users under one django model for our database. This was done in order to simplify querying. The “living_situation” field was removed from the Senior user type since we decided not to differentiate between seniors living in their own homes versus ones living in care homes for this iteration of the application. The “gender_preference” field was removed from the Activity model for the same reason. If we were to update this application in the future, we may add these fields back in order to let seniors living in care homes join activities that are offered by their care home or only request volunteers of their preferred gender, but due to the time constraints of this project, it was not possible for us to implement these features. We did, however, add a new entity: Notification. This is an improvement on our previous design and allows seniors and care homes to know when a volunteer has replied to their requests and lets volunteers know if a senior or care home has approved their request or not. Improvements were also made to the Match model with fields that allow a user (of any type) to cancel their request/participation in an activity if circumstances arrive where they can no longer make it or this activity is no longer required.



Relationships:

A “User” can be 1 of 3 types.

A “User” can receive 1 or more “Notifications” (joined on username = user).

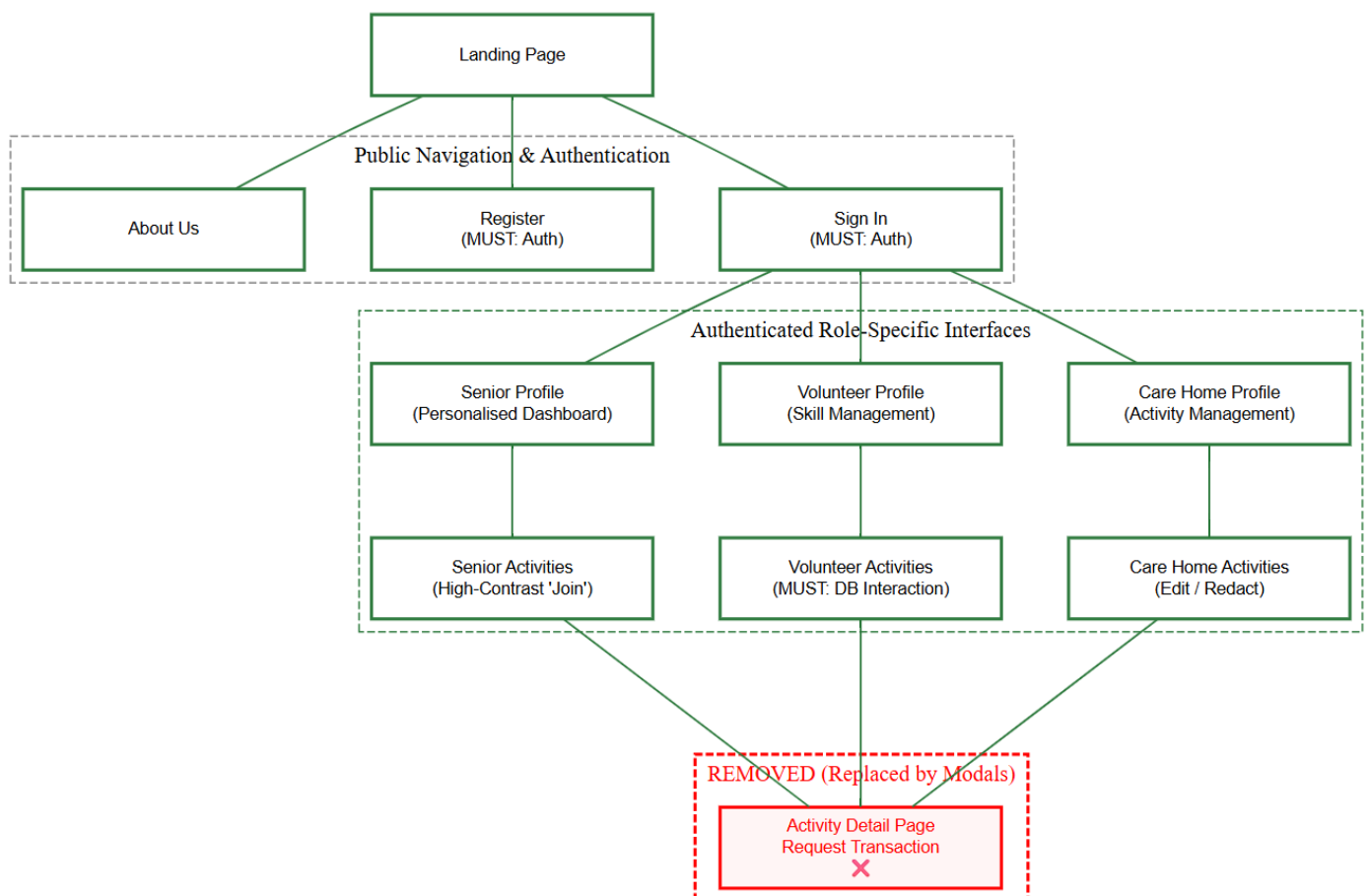
A user of usertype = “Senior” can have 1 or more “Emergency Contacts” (joined on username = user).

A user of usertype = “Senior” or usertype= “Care Home” can request 1 or more “Activities” (joined on username = requester).

A user of usertype = “Volunteer” (joined on username = volunteer) can accept 1 or more “ActivityT” requests (joined on activity_id) to be “Matched” with the requester of the activity.

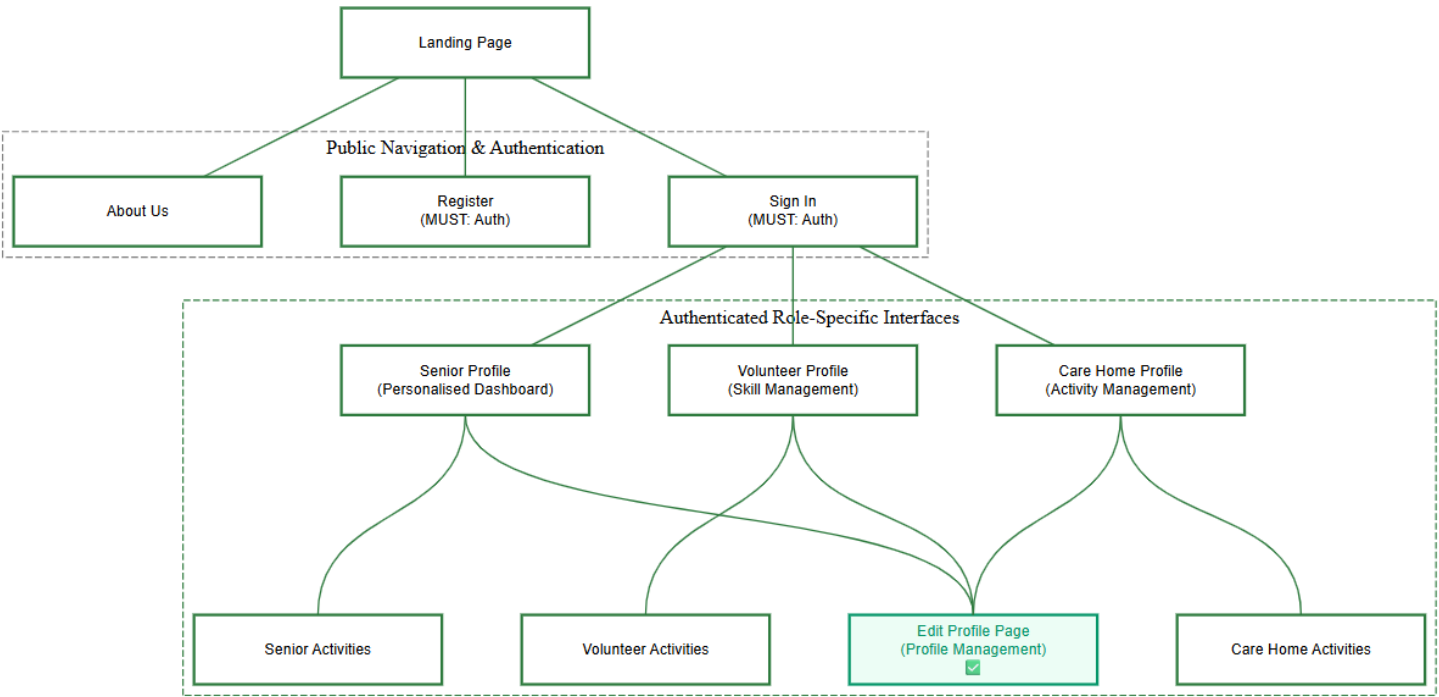
IV.) Sitemap:

Old Sitemap (Activity Details removed)



For our website we changed activity details page to a modal instead where user can check details about an activity and immediately follow up with an application rather than navigating to and from the Activity details page which would create unnecessary navigation overhead.

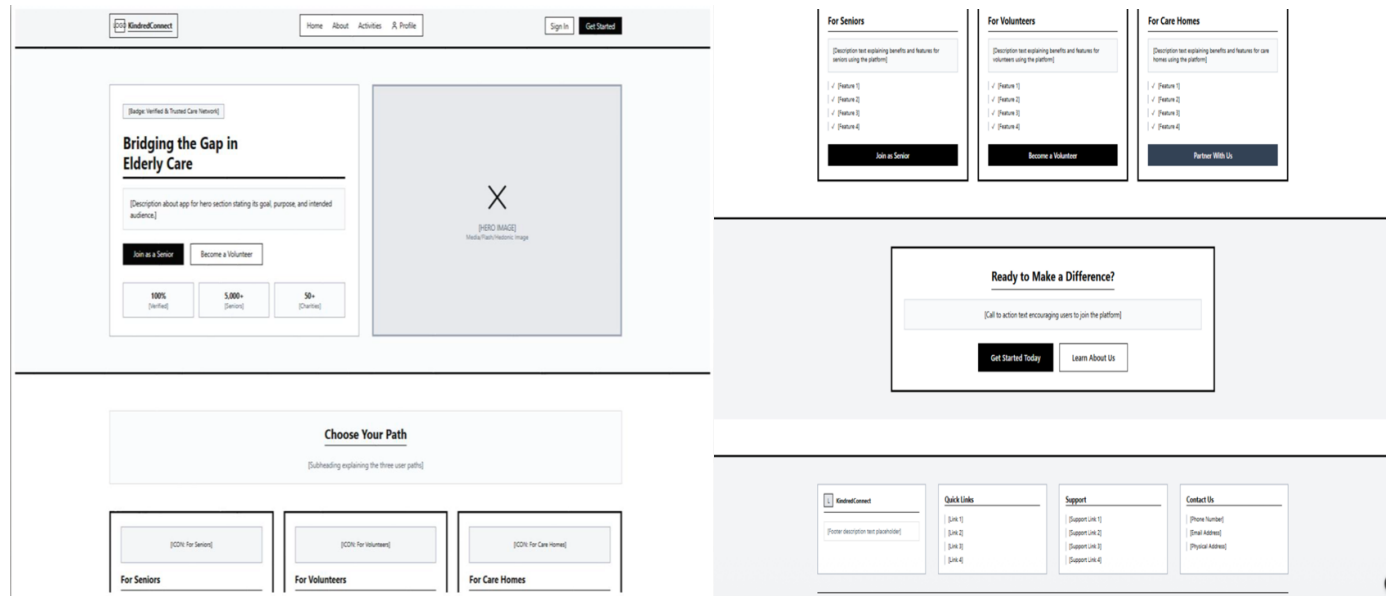
New Sitemap (Edit Profile page added)



We added Edit profile functionality on a User’s Profile (a button) on clicking which leads to an Edit Profile page where users can fill a form with updated details and submit to change their profile details.

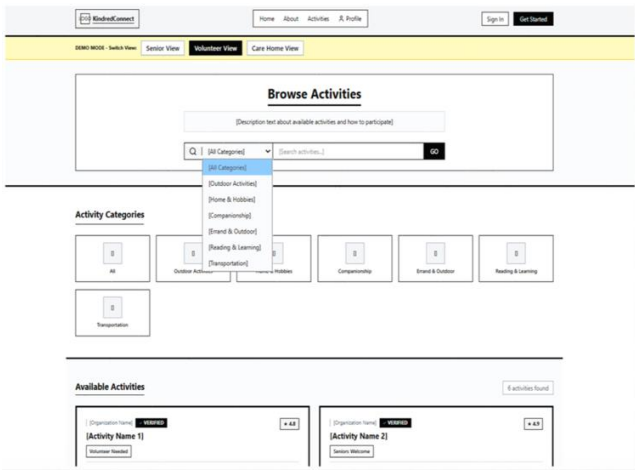
V.) Wireframes:

Landing Page

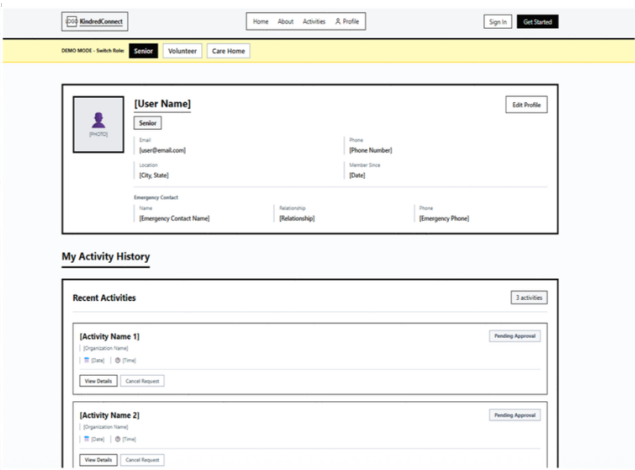


The Landing Page features a clear Hero Section and three distinct call-to-action (CTA) blocks for Seniors, Volunteers, and Care Homes. This ensures that users from different backgrounds can find their respective portals within seconds.

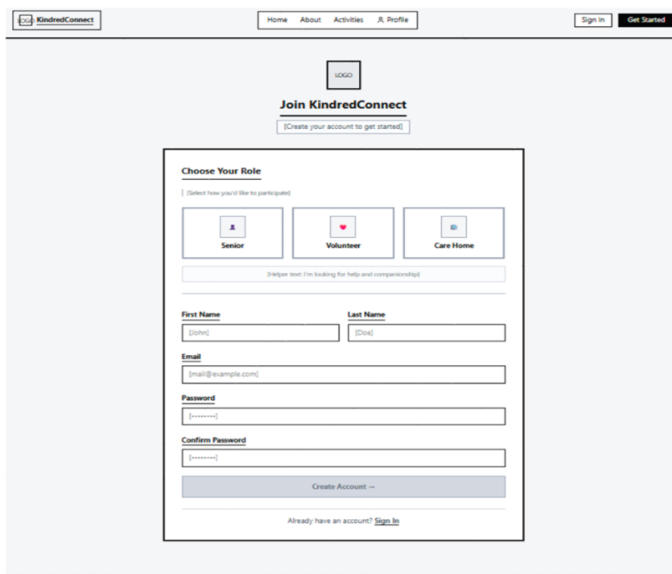
Role-Based Activities Dashboard



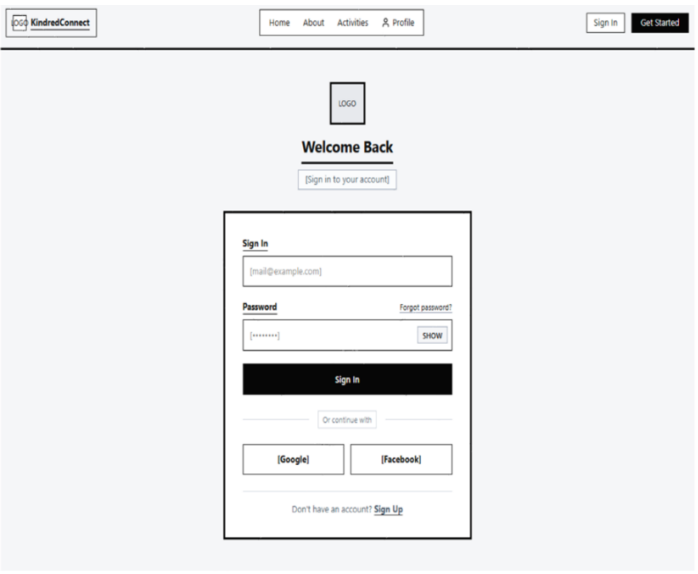
Role-Based Profile Page



Create new account



Login Page



The following wireframes outline the core user interface of Kindred Connect, focusing on accessibility for seniors and intuitive navigation for volunteers and care managers.

Implementation Highlights (2 Pages Max):

System Architecture & Core Functionality

Our backend is driven by Python and the Django framework, utilizing the Django ORM for robust database interactions. Building upon the three user roles outlined in the introduction, we implemented a custom user model and authentication system that tailors the interface and permissions to each session. Care Homes and Seniors can dynamically post new activities (CRUD operations: Create, Read, Update, Redact), while Volunteers can browse, search, and apply. Additionally, we implemented a real-time notification mechanism

that triggers updates when an application is submitted, approved, or withdrawn, ensuring seamless communication.

Front-End Interactivity & Dynamic UI (Client-Side Rendering)

To meet the requirement of going beyond static server-rendered pages, we heavily utilized JavaScript and the Bootstrap 5 framework to create a highly interactive experience.

- **Dynamic Modals & Data Injection:** Instead of forcing page reloads to view activity details, we engineered a unified modal system. When a user clicks "View Details" or a profile avatar, JavaScript dynamically fetches the specific object's data (via HTML5 data-* attributes or local arrays) and injects it into the DOM in real-time.
- **Complex State Transitions:** We implemented seamless modal interactions, such as transitioning directly from a "Volunteer Confirmation" modal to a "Review Details" modal using Bootstrap's native data-bs-dismiss and data-bs-toggle attributes, preventing UI freezing or background overlaps.
- **Asynchronous Interactions:** The platform features dynamic category filtering on the Activities page and a fully interactive Notification dropdown that users can manage and clear without refreshing the browser, alongside AJAX-driven form validations (as detailed in the Testing report).

Polished Look and Feel & Responsiveness

KindredConnect was designed with a modern, clean, and intentional aesthetic.

- **Consistent UI/UX:** We used a cohesive color palette (emerald greens, warm beiges, and clean whites) combined with intuitive bootstrap-icons. Activity feeds and user profiles are displayed using rounded cards (rounded-4) with soft shadows (shadow-sm) and subtle hover effects to create a clear visual hierarchy.
- **Full Responsiveness:** By leveraging Bootstrap's grid system (col-md-4, container-fluid), the entire application—from the split-layout authentication pages to the complex Profile dashboards—is fully responsive. It functions flawlessly across mobile, tablet, and desktop devices, ensuring the UI remains intact regardless of screen size.

Code Quality, DRY (Don't Repeat Yourself) Principle & Separation of Concerns

We strictly adhered to best practices in software engineering to ensure maintainability and scalability, which directly contributed to our high sustainability scores.

- **Template Inheritance & DRY:** We utilized Django's template engine by creating a master base.html file that houses global navigation, footers, and globally accessible modals. By populating these global modals dynamically via JavaScript, we eliminated the need to render hidden HTML modals for every single activity in a Django {% for %} loop, significantly reducing HTML bloat and server payload.
- **Separation of Concerns:** Our custom CSS styles and JavaScript logic are strictly separated from the Django templates into dedicated static files (custom.css and activities.js). This keeps our views clean, improves catchability, and allows for the targeted minification mentioned in our Sustainability Report.

Testing (1 Page Max):

This website was tested with a mixture of automated and manual tests. The automated tests are unit tests that were created in “tests.py” inside of the “kindred” directory. To run them, navigate to the project root directory (kindredcareconnect) on your computer’s terminal, and type the following command: “python manage.py test”. You should see the following output, indicating that all tests passed:

```
(kin) jocelynsickler@MacBook-Pro-2 kindredcareconnect % python manage.py test
Destroying test database for alias 'default'...
(kin) jocelynsickler@MacBook-Pro-2 kindredcareconnect % python manage.py test
Found 18 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 18 tests in 1.077s

OK
Destroying test database for alias 'default'...
```

To look at the tests themselves, you can open the “tests.py” file in a code editor. There, you will see the tests broken down into the following three categories: Model Tests, Form Tests, and View Tests. The “Model Tests” test the database by creating a sample object, ensuring that the mandatory fields are all filled out, and that the string representation of the model is working as expected. There are tests for all five models (UserProfile, EmergencyContact, Activity, Match, and Notification).

There is only one “Form Test” for the Emergency Contact form, since this was the only form created using django’s built-in form module. It tests the validity of the form data. The Sign Up form was validated through Javascript and can be tested manually by trying to submit a form that has not been filled in or where the username was already taken, where the password is less than 8 characters (or does not contain a capital letter, number, and special character) or does not match the password entered in the “confirm password field”, or the email is invalid. The username availability is validated through AJAX requests to the database, while the email and password fields are validated by matching user input against regular expressions. A Javascript regular expression is also used to validate the “Mobile Number” field in the Emergency contact form. The error for this can also be tested for manually by entering a phone number that is less than 11 digits or made up of something other than numbers. Manual testing of the Sign Up form will also show different fields if a different user type is chosen (e.g. Care Home Name vs First and Last Name). The login form was also manually tested to ensure that a user cannot login with an incorrect username or password. In this case, the page will re-render with a message from the “signin” function from the “views.py” file.

The activity posting and matching was also tested manually to ensure that only usertypes “Senior” or “Care Home” can post or approve of an activity, and only “Volunteers” can sign up for one.

Going back to automatic tests, there are a number of “View Tests” that test the proper response and template use of the “index” page and tests that ensure only a logged-in user can access the “profile” page.

Accessibility Report (1 Page Max):

Our original accessibility plan is as follows. We have since revised and improved upon it, and have listed our improvements and specific implements below each bullet point.

- (WCAG Principle 4 - Robust, Guideline 4.1 - Compatible, 4.1.3 - Status Messages) Status messages will be displayed when a user tries to submit a form (i.e. registers for an account or signs in). These messages will be enclosed in tags marked `<div role="status" aria-atomic="true">Status message here.</div>` so that they can be read aloud to users who are viewing the app with screen readers.

Improvements and Implementation: Aria status messages were added to the Signup and Emergency Contact forms to notify them of missing fields and submission errors. **Aria-labels** were attached to all buttons and the activity search field so their contents can be read by screen readers. The **“for” attribute was also added to form labels** so they can be read aloud.

- (WCAG Principle 1 - Perceivable, Guideline 1.4 - Distinguishable, 1.4.3 - Contrast) The app design should be clean and uncluttered with simple fonts and high contrast between elements, font colours, and backgrounds for better legibility. The design will be run through a color-blind filter to check for problems.

Improvements and implementation: Font sizes were changed to rem to ensure the user can easily scale the text based on their preferences. **The default root font-size (1rem) was set to a large 18px** with paragraphs and similar text elements set to 1rem and the largest heading (h1) set to 2.5rem (45px). The website design features a **high contrast design** with black **sans-serif** text against a beige or white background and bold button text against a contrasting background (either light vs dark or vice versa) for easy legibility. Every page was run through **protanopia, deuteranopia, and tritanopia filters** to check the colours for colour-blind users.

- (WCAG Principle 2 - Operable, Guideline 2.1 - Keyboard Accessible, 2.1.1 - Keyboard) Forms and links must be operable using either the keyboard or the mouse. Standard HTML controls must be used for all forms and link elements to ensure that users with assistive technologies can fill out forms and navigate the app successfully using the keyboard.

Implementation: All forms (login, signup, activity, emergency contact) were tested to check that users can navigate between fields using only the tab key on the keyboard.

- (WCAG Principle 3 - Understandable, Guideline 3.1 - Readable, 3.1.5 - Reading Level) All content on the app must be written at a lower secondary school reading level or lower. Descriptions should be short and use common, easily understandable words. The aim is to make the app accessible to those with cognitive/language impairments as well as those with physical disabilities who may become fatigued by the extra scrolling that often accompanies overly wordy passages.

Implementation: The activity dashboard was deliberately kept simple and only features five category buttons (All, Outdoor Activities, Home & Hobbies, Companionship, Errand) to reduce reading fatigue.

- (WCAG Principle 1 - Perceivable, Guideline 1.1 - Text Alternatives, 1.1.1 - Non-text Content) All images must be accompanied by a descriptive "alt text" that explains what the image is of for people who are visually impaired. In addition to this, images alone must never be used to communicate any vital information or functions.

Implementation: Images only appear on the about page, and each one is accompanied by alt text, so visually impaired users know who or what is in them.

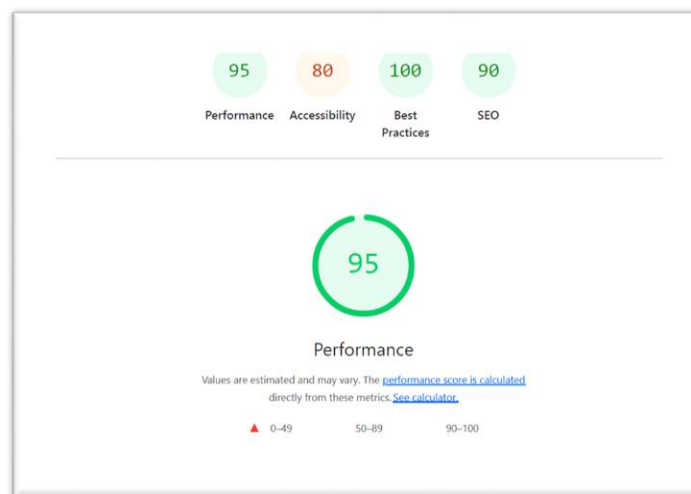
Sustainability Report (1 Page Max):

For KindredConnect, we used Google Lighthouse to audit KindredConnect, testing all the pages. Our goal was to show how technical optimizations directly lead to resource optimization.

Improvement 1: Performance & Environmental Baseline

Our initial test served as the baseline which we performed in Incognito Mode. Even with a solid performance score of 95, we identified several hidden energy costs:

- **Data Waste:** The site was carrying 192 KiB in inefficient cache data and several KiB in redundant CSS and JavaScript.
- **Processing Latency:** Render-blocking requests were adding 720 ms of waiting time. On a mobile device, this means the processor stays at high power for longer than necessary.
- **Baseline Impact:** These issues meant that for every user visit, we were sending more data than needed and forcing the user's device to work harder to render the page.

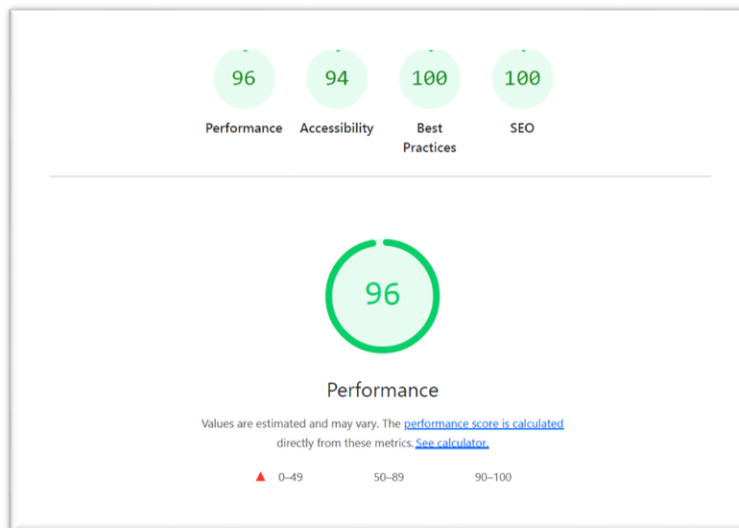


Improvement 2: Accessibility Optimization

In the second phase, we implemented specific code fixes. This resulted in a performance bump to 96 and a major accessibility jump from 80 to 94.

- **Accessibility:** By fixing button names, heading orders, color contrast and adding arial labels we improved the site's Social Sustainability. Making the site work better for assistive technology means users can find what they need faster, reducing overall time-on-page and total energy use.
- **Resource Savings:** We cut down unused CSS by 47 KiB and optimized image delivery to save another 12 KiB.

- **Efficiency Gains:** We reduced the total number of long main-thread tasks from 5 down to 4, which lowers the CPU intensity on the user's phone or computer.
- **Metadata:** Implementing a missing meta description successfully raised our SEO score from 90 to 100. This improves the precision of search engine indexing, ensuring users reach their destination with fewer redundant queries.



Summary of Improvements:

Metric	Improvement 1	Improvement 2	Result
Performance	95	96	Leaner Code
Accessibility	80	94	Better Navigation
SEO	90	100	Optimized metadata
Unused CSS savings	15 KiB	47 KiB	Reduced data load

By moving from our baseline to Improvement1 to Improvement 2, we’ve ensured KindredConnect is not only faster for our users but also more energy-efficient and inclusive for the whole community.

Appendix (1/2 Page Max):

I.) Team Contributions:

Aasim	Xue	Jocelyn
Sustainability Report (and implementation of changes)	Implementation Highlights	Accessibility Report (and implementation of features)
Sitemap	User Stories	Skeleton Django site

System Architecture	Introduction	Testing
Diagrams	Database Models	About Page
Index Page	Profile page styling and views	Login in and Sign Up Pages
Base html page and CSS styles	Bug fixing	Emergency Contact functionality
Activities		Bug fixing
Bug fixing		
Deployment		

II.) AI use statement:

AI was used to generate regular expressions and for help with debugging.